Project Report: FairShare

Team Members

Instructions: List the names of the team members.

Content Box:
Brennan Long, Elias Villalobos, Brandon Aitken, Jensen Trempe, Cole Wallace Wayne

---

Project Description

Instructions: Provide a comprehensive overview of the FairShare web application.

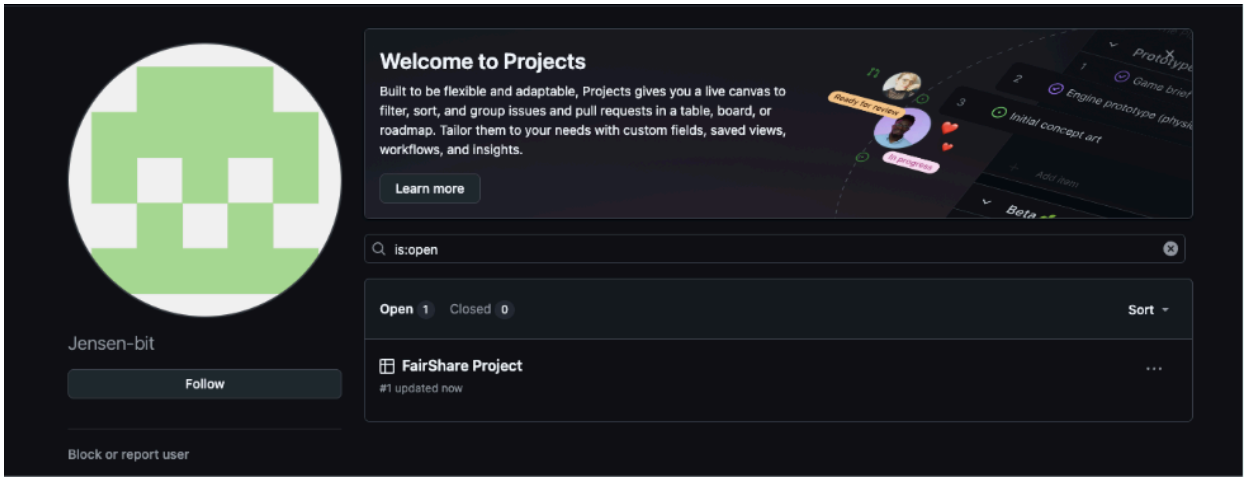
Content Box:
The FairShare web application can be described as a full stack project that allows groups of people like friends, roommates, and others, to easily track expenses and manage money. In addition to logging expenses for groups, this application also automatically splits bills to show how much each member of the group is owed and owes in the form of shares for each individual. The application utilizes authentication through hashing of passwords and session management to allow users to interact with their data securely and privately. The backend is powered by the Node.js/Express and all information about users, groups, expenses, payments and other important information is stored in a database running PostgreSQL. The front-end consists of several Handlebars templates. Other important functions of this application include: group invite through token, a page for viewing payment history, a notification system for informing members about pending invitations, and a way to submit balance requests to be approved by the manager of the group. In general, this application offers a great solution for those who want an organized and accurate way of managing group expenses and payments.

Project Links

Instructions: Provide links to the Project Tracker, Video, Version Control System (VCS), and Deployment.

Content Box:
Project Tracker - GitHub project board: https://github.com/users/Jensen-bit/projects/1
Video:
https://drive.google.com/file/d/1OD-3krRmsdxoGni-ZKkXLquwxYU8Z7Zz/view?usp=sharing
VCS: https://github.com/Jensen-bit/FairShare

Deployment: https://fairshare-asop.onrender.com/



Contributions

Instructions: Write a brief (not more than 100 words) from each team member about their contributions. This should include the technologies worked on and features contributed to. Optionally, include a screenshot of the project Board and a screenshot of the contributions on GitHub.

Content Box:
Brennan Long: Throughout this project, I worked on balances, payment histories, and pay balances, along with various bug fixes and improvements to the website. I used JavaScript for coding and had to work with the database we created. Although implementation is relatively straightforward for these topics, several issues arose, such as balances going into negatives, rounding errors with JavaScript (which was changed to ensure all calculations happen through PostgreSQL), and ensuring payments work correctly across user groups. I was also the person responsible for setting up and troubleshooting Render.

Elias Villalobos: My primary focus was on front-end development, specifically implementing a dynamic dark theme feature. This feature automatically adapts the website's theme (light or dark) based on the user's operating system preferences, which requires significant research and problem-solving. The implementation utilized JavaScript and CSS. This customization enhances the user experience by providing a seamless, preference-matching visual interface. Additionally, I worked on resolving bug fixes and managing merge conflicts throughout the development process.

Brandon Aitken: As a backend developer, I built the database using PostgreSQL to ensure efficient data updating and retrieval. I handled the integration of the Nodemailer API, enabling important features such as automated group member authentication and system notifications. I wrote some JavaScript for email methods and expense creation Additionally, I managed our Docker environment and Git workflow to maintain development consistency across the team,

creating wireframes to express our vision. My focus was on building a scalable server that bridged our data storage with external communication services.

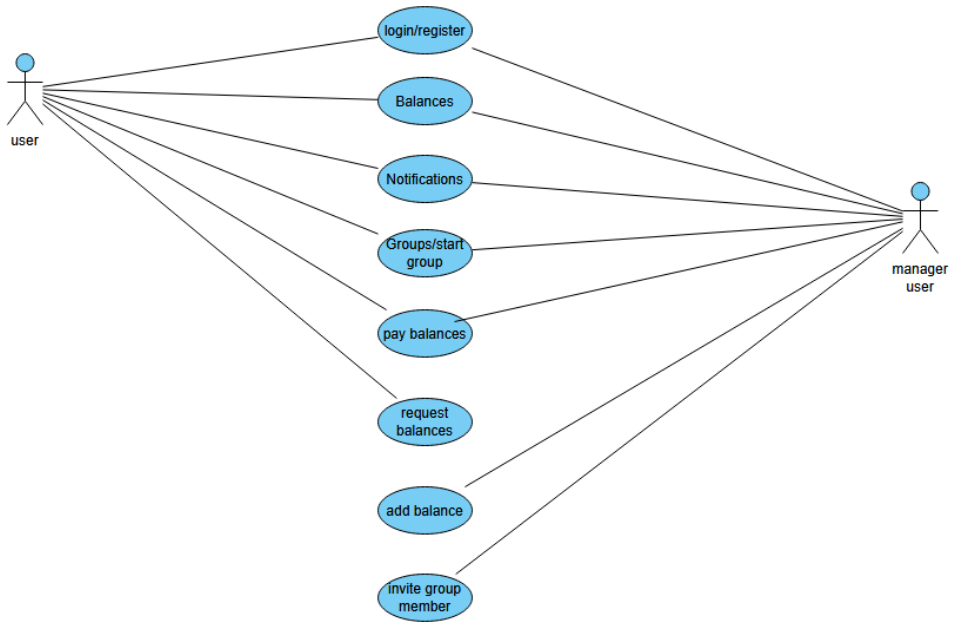
Jensen Trempe: My role in this project was mainly as a front end developer. I was the lead for the styling (CSS) of the website. For this, I used Bootstrap and AdminLTE as our framework for the sidebar and navigation bar. This allowed me to build a consistent fluid theme across all the pages of the website. I also worked on implementing handlebars in our project for easy and great reusability across each page. In addition, I was responsible for the individual layout of each page. Finally, I contributed to making our test cases using Mocha / Chai in order to test our code and to ensure it functioned as intended.

Cole Wallace Wayne: I worked in a hybrid role on this project, contributing to both the frontend and backend rather than focusing on a single area. On the backend, I used Node.js and SQL to develop core API endpoints and manage data relations within our PostgreSQL database. On the frontend, I implemented UI components using JavaScript and CSS to ensure seamless data integration. Additionally, I helped manage our Docker containerization to maintain environment consistency across the team and assisted in troubleshooting full stack integration issues to ensure a cohesive final product.
Insert Contributions from all members

Use Case Diagram

Instructions: Include a use case diagram for your project. You can build on the use case diagram you created in the proposal. If you built a complete use case diagram for the proposal, you can include it as is.

Content Box:

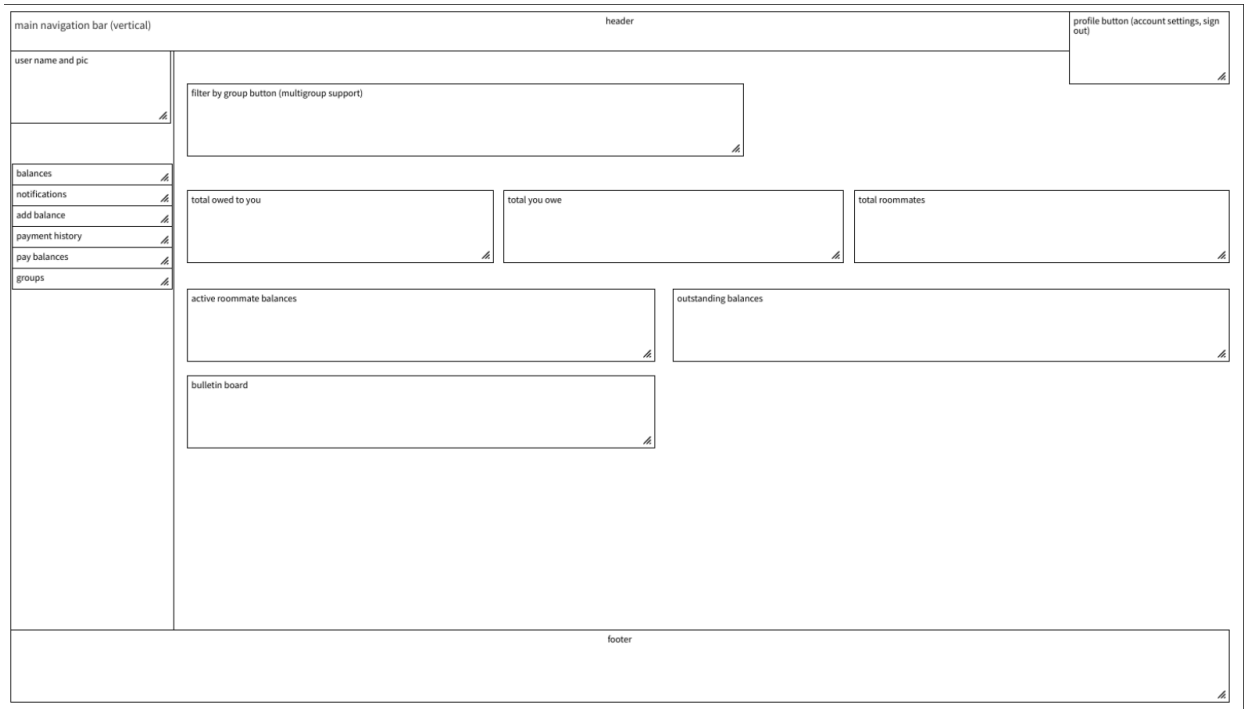


Wireframes

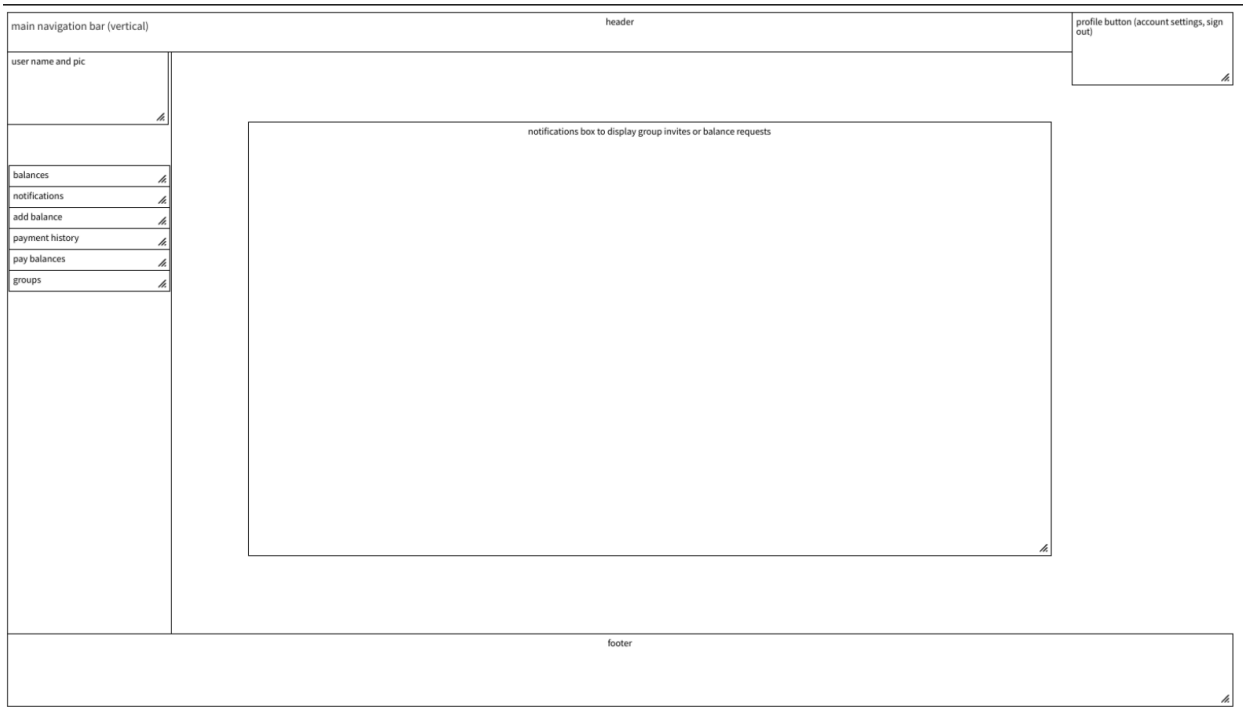
Instructions: You must include all the wireframes that were created for the project. It is expected that you have at least one for each page. They can be photographs of hand-drawn images.

Content Box:

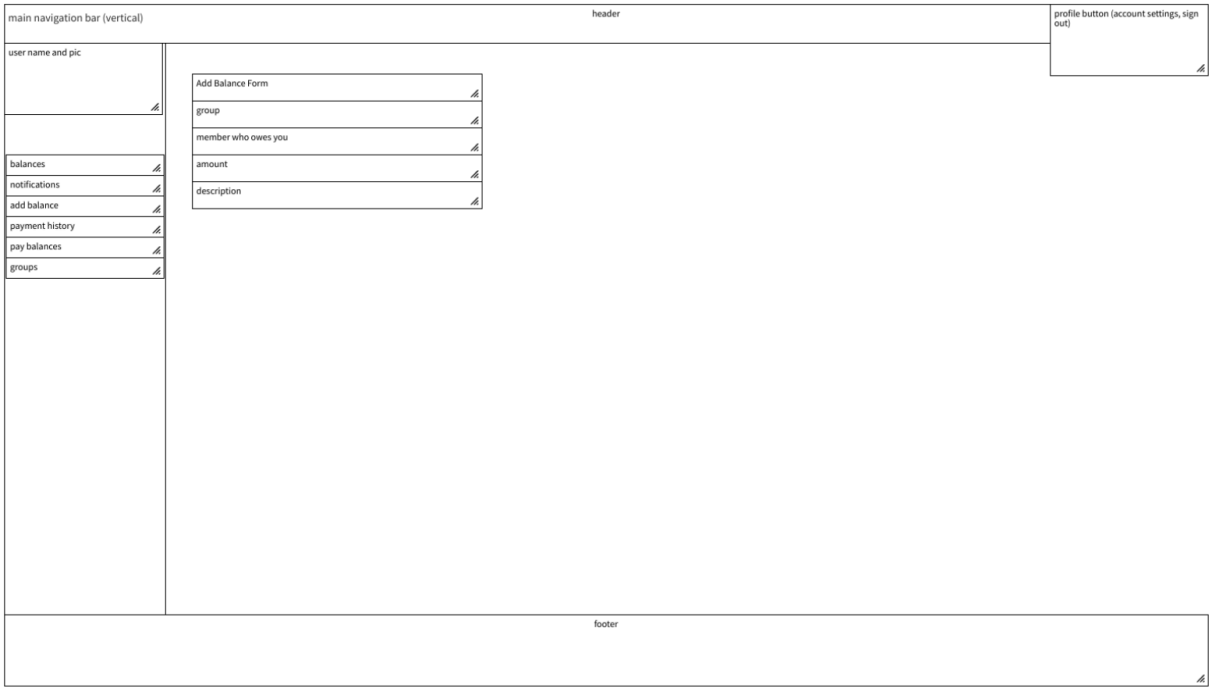
Home Page:



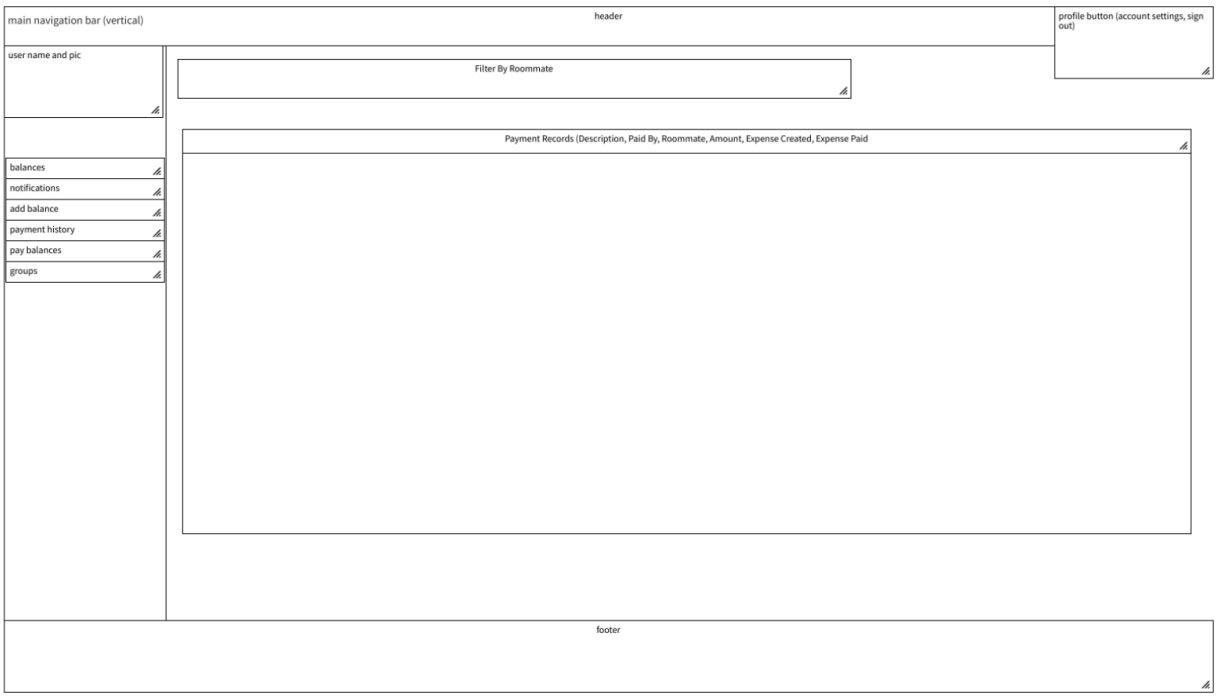
Notifications Page:



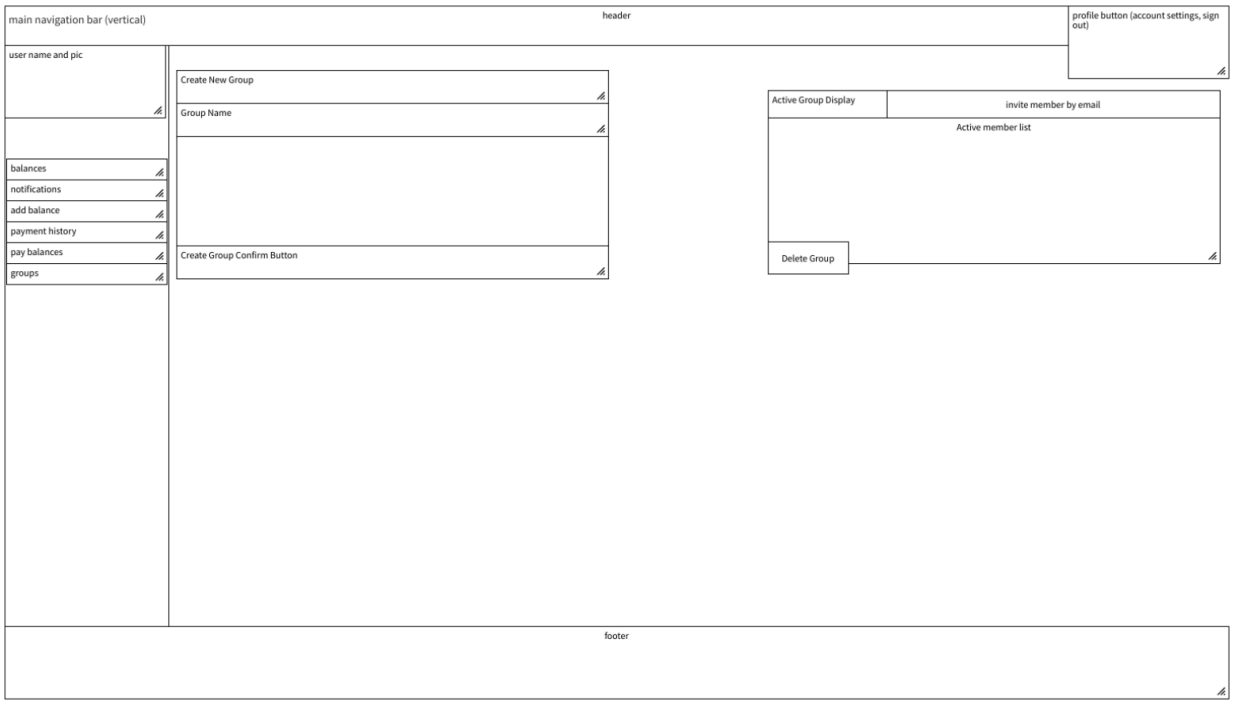
Add Balance Page:



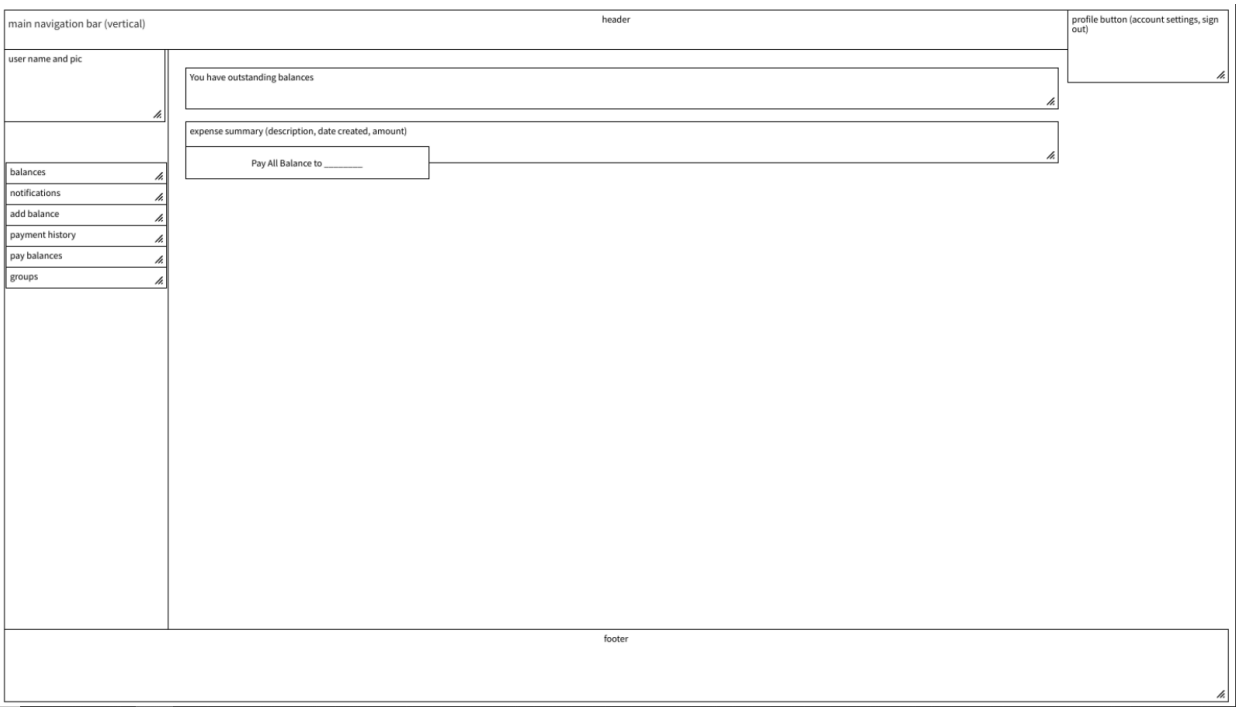
Payment History Page:



Groups Page:



Pay Balances Page:

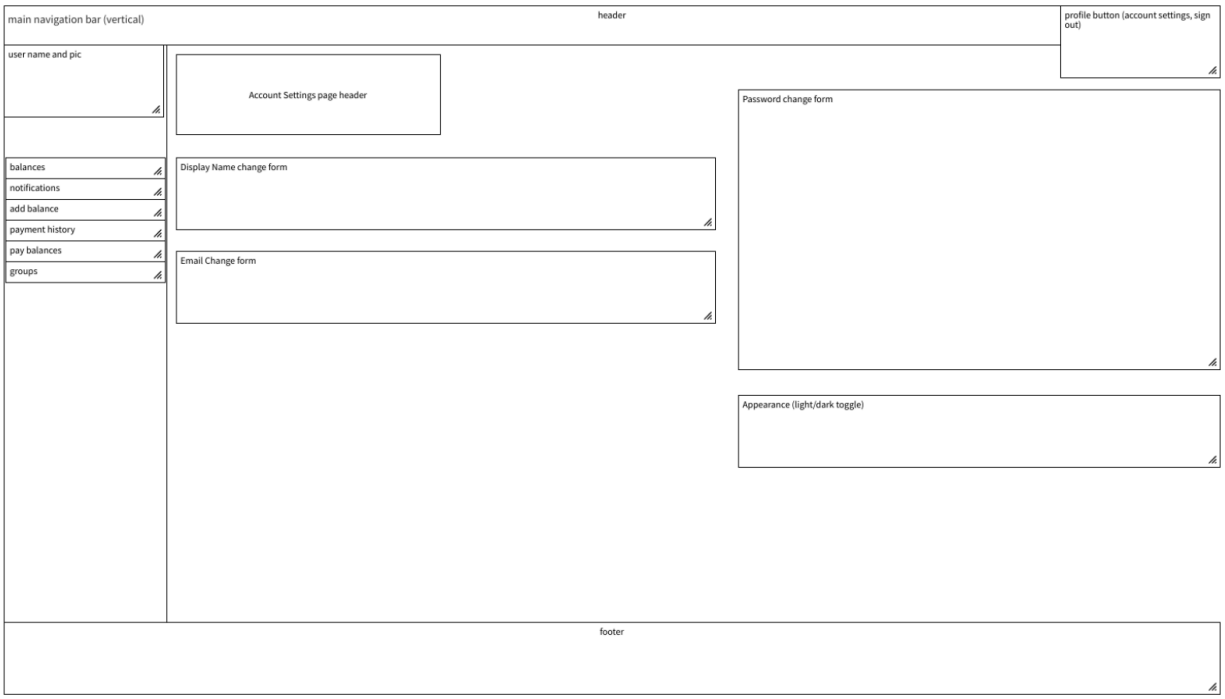


Login Page:

FairShare Header
Sign In to manage your expenses
email
passsword
Sign In
Register

Register Page:

Account Settings Page:



Test Results

Instructions: In Lab 10, you created a Test Plan. You need to include the test results and observations in the project report. Refer to this link for more information.

Content Box (Test Results and Observations):
We conducted two types of tests in order to evaluate our application, namely, automated back-end tests and user acceptance tests.
Automated Test Cases:

- **Welcome Route Test:** Expected a 200 status code and a JSON response with the message Welcome!
- **Register Route - Positive Test:** Expected a 200 status code and a success message.
- **Register Route - Negative Test:** Expected a 400 status code and an error message saying that name, email, and password are required.
- **Balances Route Test:** Expected a 200 status code and HTML containing Roommate Balances for an authenticated session.
- **Mark Paid Negative Test:** Expected a 400 status code because the backend should reject invalid non-numeric IDs.

Observations from User Acceptance Testing:
During the testing, users were asked to register, log-in, see their balances, and interact with other features. The majority of users were able to easily follow the expected flow and comprehend the key purpose of FairShare as providing a simple means for splitting shared expenses.
One observation is that users expected visible feedback after performing an action (e.g., registering or marking an item as paid), reinforcing the need for confirmation messages. Another indicated that users wanted their balances to be easy to read at a quick glance, leading to a focus on making the balance page simple and direct.
Testing confirmed that the important back-end routes work correctly and that users are able to understand how to navigate through the core features of the application.